

Peer Review

Grupo revisor: 4

Nome: Ana Clara Stolses
NUSP: 15654835

Nome: Isabela Lima
NUSP: 15678780

Grupo revisado: 5

1. Qualidade do Projeto:

a) Requisitos:

O README apresenta as funcionalidades principais, mas o repositório não traz um documento de requisitos dedicado. Também não há evidência de rastreabilidade entre requisitos e a implementação.

b) Descrição do Projeto:

A descrição do projeto no README é clara sobre o objetivo e a arquitetura geral. No entanto, algumas afirmações são mais amplas do que o sistema entrega, como a sugestão de gráficos no painel de vendas.

c) Comentários sobre o Código:

O código possui comentários e alguns blocos de Javadoc, mas a cobertura é irregular. Há classes e métodos sem documentação, e a separação entre interface e lógica não está totalmente consolidada.

d) Plano de Testes:

Existe uma suíte de testes em TestRunner.java e o README descreve o que é testado. O plano de testes não está formalizado em um documento separado e depende de testes independentes do framework padrão.

e) Resultados dos Testes:

Os testes verificam funcionalidades de persistência, relatórios e lógica de pedidos. São úteis, mas ainda limitados e não cobrem cenários de falhas, concorrência ou casos de borda mais complexos.

f) Procedimentos de Build:

As instruções de compilação e execução estão presentes e são simples (javac *.java e java Main). Seria melhor incluir um script ou ferramenta de build para reduzir erros manuais.

g) Problemas, Qualidade Geral e Outros:

O trabalho está visualmente consistente e funcional, mas há problemas técnicos importantes: uso do pacote padrão, forte acoplamento entre UI e modelo de dados, e persistência em CSV sem abstração adequada.

f) Interações com o Grupo:

Não houve interações diretas com o grupo.

Avaliação de Projeto: 4/5 estrelas

2. Qualidade do Código:

a) Design do Código e uso de frameworks:

A solução usa Swing e persistência CSV básica. Não há uso de frameworks de teste, nem de gerenciamento de dependências mais moderno.

b) Organização do Código:

Os nomes das classes são razoáveis, mas todas estão no pacote padrão. A organização poderia ser melhor com pacotes separados para modelo, visão, lógica e persistência.

c) Funcionamento do Código:

O sistema funciona para as operações básicas de pedido, estoque e vendas. Entretanto, a lógica depende muito de dados lidos diretamente de tabelas e de parsing de strings, o que fragiliza a manutenção.

d) Documentação do Código:

Há documentação parcial em algumas classes, mas não é suficiente. Métodos de GUI e de lógica não documentados devem ser melhorados para facilitar a leitura e a manutenção.

Avaliação de Código: 3/5 estrelas

3. O que faríamos diferente:

Eu recomendaria organizar o projeto em pacotes, separar claramente os modelos de domínio dos componentes de interface e usar classes específicas para produto, transação e estoque. Também sugiro usar BigDecimal para valores monetários, adotar JUnit para testes e implementar uma camada de persistência menos dependente de tabelas e arquivos CSV.

Comentários adicionais:

- Seria útil ter um documento de requisitos e um plano de teste formalizados.
- A interface está bem apresentada, mas a lógica de negócio deve ser menos acoplada aos componentes Swing.